الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

College of Computing and Informatics

Web Programming

IT361

Project

Phase1 - Deadline: Sunday 26/07/2026 @ 23:59

Phase2 – Deadline: Sunday 09/08/2026 @ 23:59

[Total Mark is 20]

Student Details: CRN:

Name: Ruba Almuzaini **ID:**
Name: Lamya Alotaibi **ID:**
Name: Manal Almusabbihi **ID:**
Name: **ID:**

Instructions:

- You must submit two separate copies **(one Word file and one PDF file)** using the Assignment Template on Blackboard via the allocated folder. These files **must not be in compressed format**.
- It is your responsibility to check and make sure that you have uploaded both the correct files.
- Zero mark will be given if you try to bypass the SafeAssign (e.g. misspell words, remove spaces between words, hide characters, use different character sets, convert text into image or languages other than English or any kind of manipulation).
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented. This includes filling your information on the cover page.
- You must use this template, failing which will result in zero mark.
- You MUST show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- Late submission will result in ZERO mark.
- The work should be your own, copying from students or other resources will result in ZERO mark.
- Use **Times New Roman** font for all your answers.

Learning Outcome (1, 2,3, 5,6):

Describe methods and tools in web development.

Create web pages using HTML5 and CSS3.

Develop dynamic web pages using JavaScript.

Create Rich Internet Applications.

Build web applications using PHP and MySQL.

Description and Instructions

Project Idea: “Campus Events Hub”

Each group will design and build a small dynamic website for a fictional university club or department that publishes upcoming campus events (workshops, seminars, competitions, trips) and lets students register for them online. The site is built with HTML and CSS for the front end, and basic PHP for the dynamic parts (page templates, form handling, validation, and saving registrations).

Group size: 3–4 students. **Duration:** 6 weeks.

Functional Requirements (Minimum)

The website must contain at least the following pages:

- **Home page:** introduction to the club/department and highlights of the next 3 upcoming events.
- **Events page:** a list of all events, generated by PHP from a data source (PHP array, text/CSV file, or a simple database table — the group chooses).
- **Event details page:** one page that displays the details of a selected event (using a GET parameter, e.g. event.php?id=3).
- **Registration page:** an HTML form (name, student ID, email, selected event) processed by PHP with server-side validation; valid registrations are appended to a text/CSV file or database table, and a confirmation message is shown.
- **Registrations list page:** a page that reads the stored registrations and displays them in an HTML table.
- **About / Contact page:** team members and a simple contact form (validation only; no email sending required).

Technical Requirements

HTML

- Valid, semantic markup (header, nav, main, section, footer); correct use of headings, lists, tables, images, and links.
- At least one data table and at least two forms with appropriate input types and labels.

CSS

- One shared external stylesheet (no inline styles except where justified).
- A consistent color scheme and typography, a styled navigation bar, and hover/focus states.
- A basic responsive behavior (e.g., layout adapts on small screens using media queries or flexible units).

PHP

- Shared header and footer using include/require on every page.
- Handling of GET and POST data with server-side validation (required fields, email format, etc.) and clear error messages.
- Reading and writing data (text/CSV file or a simple database table) and displaying it with loops.
- Code is organized, indented, and commented; no frameworks or site builders are allowed.

Timeline

- **Week 1:** team formation, choosing the club theme, sitemap and page sketches.
- **Weeks 2–3:** static HTML/CSS version of all pages.
- **Week 3 (end):** submit the mid-project progress report.
- **Weeks 4–5:** PHP work: templates, forms, validation, data storage and display.
- **Week 6:** testing, polishing, final report, and submission.

Submission Guidelines and Instructions

1. Mid-Project Progress Report

Each group must submit a short progress report at the end of Week 3, which must include:

- Work completed this period.
- One key decision made during this period.
- One problem encountered.
- Plan for the next period.
- Each group member's input/participation.

2. Weekly Progress Log (Inside the Submission File)

Students must document their progress **inside the final submission file itself**, in a dedicated section named **“Progress Log”**. The log must contain one dated entry per week (minimum 5 entries) written at the time of the work, not at the end. Each weekly entry must include:

- **Date and week number.**
- What was accomplished that week (2–4 sentences).
- Which member did what.
- At least one screenshot of the website or code as it looked that week (screenshots must show a visible date/timestamp, e.g., the system clock or file properties).

What NOT to do: Students must not build the entire project at the last minute and then write the log retroactively. Entries that all look written in one sitting, screenshots without dates, or a log that does not match the final work are treated as red flags regardless of the quality of the final website.

3. Project Artifacts

A separate section in the final report must be named “Project Artifacts” and must include:

- The full source code of the website as a ZIP file (all .html, .css, .php files, images, and any data files), submitted together with the report.
- A short “How to run” note (e.g., which folder to place in XAMPP/htdocs and which page to open first).
- The Progress Log section described above, with its screenshots.

4. Written Reflection (End-of-Project Report)

The final report must end with a Reflection section (300–500 words), written in the students' own words, addressing some of the following:

- Describe one specific challenge or obstacle you encountered during this project and explain in detail how you identified and resolved it.
- Explain why you chose your specific approach (e.g., CSV file vs. database, page layout technique) over at least one alternative you considered, and what tradeoffs informed that decision.
- If you used any AI tools during this project (e.g., ChatGPT, Claude, Copilot), disclose which tool(s), for what specific purpose (e.g., debugging, grammar checking, brainstorming), and how you verified or modified the output.
- What would you do differently if you were to redo this project with more time or resources?

This section will be evaluated for specificity and consistency with the technical content of the project report.

Important Notes

- **AI use:** Students may use AI tools (ChatGPT, Claude, Copilot, Gemini, etc.) only for brainstorming approaches, debugging small errors, or proofreading text. AI must not be used to generate the code logic or produce substantial portions of the report or website. Any permitted AI use must be disclosed in the Reflection section. Undisclosed AI use, or AI use beyond this scope (disclosed or not), will result in a **zero** grade for the entire project.
- Any submission lacking a valid, inspectable Progress Log, or showing evidence of AI generation or plagiarism, will receive a **zero** grade for the entire project.
- Failure to submit the mid-project progress report as required, or submission of reports found to be fabricated, copied, or AI-generated, will result in a **zero** grade for the entire project.

• Project Rubric (Total: 20 marks)

Component	Points
Final website (functionality, code quality, design)	10
Final report	4
Mid-project progress report	2
Reflection section	4
Weekly Progress Log (inside the submission file)	Pass / Fail
Total	**20**

Mid-Project Progress Report

Project Title: Campus Events Hub

Work Completed During This Period

During this period, we chose the Campus Events Hub idea. We decided to create a website for a fictional University IT Club. The website will show university events such as workshops, seminars, competitions, and trips. Students will also be able to register for the events online.

We selected the main pages needed for the website. These pages are Home, Events, Event Details, Registration, Registrations List, and About/Contact. We also created the basic project folders and connected the pages using a navigation bar.

We completed most of the HTML and CSS work. We created the page layouts, event cards, registration form, contact form, buttons, table, header, and footer. We also selected a consistent color scheme and started making the website responsive for smaller screens.

One Key Decision Made

One important decision was to use a PHP array for storing the event information and a CSV file for saving student registrations.

We selected CSV instead of MySQL because the project is small and does not need a complex database. CSV is easier to create, test, and run using XAMPP. It also allows us to save and read registrations using basic PHP functions.

One Problem Encountered

One problem we faced was repeating the same header, navigation bar, and footer code on every page. This would make the website harder to update because the same change would need to be made on every page.

We decided to solve this problem by creating shared `header.php` and `footer.php` files. These files can be included on every page using `require`. This reduces repeated code and keeps the website design consistent.

Plan for the Next Period

During the next period, we will complete the PHP parts of the website. We will display the events using PHP loops and complete the Event Details page using a GET parameter.

We will also complete the Registration form using POST and add server-side validation for the name, Student ID, email, and selected event. Valid registrations will be saved in a CSV file and displayed in an HTML table on the Registrations List page.

After completing these parts, we will test all the pages, fix any errors, improve the responsive design, and prepare the final report and project files.

Group Members' Participation

Ruba Almuzaini worked on selecting the project idea, preparing the sitemap, creating the Home page, and helping with the shared header and footer.

Lamya Alotaibi worked on the Events page, Event Details page, event information, and the basic PHP structure.

Manal Almusabbihi worked on the Registration page, About/Contact page, CSS design, forms, and responsive layout.

Campus Events Hub

Final Project Report

Introduction

Our project is called Campus Events Hub. It is a small dynamic website for a fictional university IT club. The website helps students find upcoming campus events and register for them online.

The website contains different types of events such as workshops, seminars, competitions, trips and technology activities. Students can view all events, open the details of a selected event and submit a registration form.

We used HTML and CSS for the front-end design and PHP for the dynamic parts. The event information is stored in a PHP array, while the student registrations are saved in a CSV file.

Project Objectives

The main objectives of this project were to create a website that:

- Shows upcoming university events.
- Displays the details of every event.
- Allows students to register for events.
- Checks the form information using PHP.
- Saves registration data in a CSV file.
- Displays saved registrations in a table.
- Works correctly on computers and small screens.

Website Pages

Home Page

The Home page introduces the University IT Club and explains the purpose of the website.

It also displays the next three upcoming events. Each event includes its title, category, date, time, location and short description. A button is available to open the Event Details page.

Events Page

The Events page displays all the events available on the website.

The event data is stored in a PHP array inside the `events-data.php` file. A PHP `foreach` loop is used to display the events.

Every event contains two buttons. The first button opens the details of the event, and the second button opens the Registration page.

Event Details Page

The Event Details page displays one selected event.

The event is selected using a GET parameter, for example:

```
event.php?id=1
```

PHP reads the event ID and searches for the event inside the array. If the event exists, the page displays its title, date, time, location, number of seats, category and description.

If the ID is missing or incorrect, the page displays an Event Not Found message.

Registration Page

The Registration page contains a form with the following fields:

- Full name.
- Student ID.
- Email address.
- Selected event.

The form is submitted using the POST method.

PHP checks the data before saving it. If there is an error, the page shows a clear error message. If the information is valid, it is saved in the CSV file.

The form also keeps the entered information when an error happens, so the student does not need to enter all the data again.

Registrations List Page

The Registrations List page reads the saved information from the CSV file.

The information is displayed in an HTML table. The table contains:

- Student name.
- Student ID.
- Email.
- Event name.
- Registration date.

A PHP loop is used to display every registration.

About and Contact Page

The About section contains information about the fictional University IT Club and the project team.

The team members are:

- Ruba Almuzaini.
- Lamya Alotaibi.
- Manal Almusabbihi.

The Contact section contains another form. It asks for the name, email, subject and message.

This form performs validation only. It does not send a real email because email sending was not required in the project instructions.

Technologies Used

HTML

HTML was used to create the structure of the website.

We used semantic elements such as:

```
<header>
<nav>
<main>
<section>
<footer>
```

We also used headings, paragraphs, links, lists, forms, labels, buttons and tables.

CSS

One external CSS file is used by all website pages.

CSS was used for:

- The color scheme.
- Navigation bar.
- Event cards.
- Forms.
- Buttons.
- Tables.
- Footer.
- Hover effects.
- Focus effects.
- Responsive design.

Media queries were used to change the layout on small screens.

PHP

PHP was used for the dynamic parts of the website.

It was used to:

- Include the shared header and footer.
- Store event information.
- Display events using loops.
- Read GET parameters.
- Process POST forms.
- Validate information.
- Save data in a CSV file.
- Read data from a CSV file.
- Display registrations in a table.

Shared Header and Footer

The website uses shared files for the header and footer.

The files are:

`includes/header.php`
`includes/footer.php`

Each page includes these files using `require`.

This reduces repeated code and makes it easier to update the navigation bar or footer.

Data Storage

The event information is stored in a PHP array inside:

`includes/events-data.php`

The student registrations are stored inside:

`data/registrations.csv`

We selected a CSV file instead of a database because the project is small and only needs basic storage.

A CSV file was easier to set up and test. It also does not require MySQL.

The disadvantage is that CSV is not suitable for a large website with many users. A database would be better for a larger system.

Server-Side Validation

The Registration form checks the following:

- Full name is required.
- Full name must contain at least three characters.
- Student ID is required.
- Student ID must contain only numbers.
- Student ID must contain between 6 and 12 digits.
- Email is required.
- Email format must be valid.
- A valid event must be selected.

The Contact form checks:

- Name is required.
- Email must be valid.
- Subject is required.
- Message is required.
- Message must contain at least ten characters.

The validation is performed using PHP on the server side. HTML validation is also used, but the project does not depend only on it.

Testing

We tested the website after completing the main functions.

The following tests were completed:

- Opening the Home page.
- Checking that three upcoming events appear.
- Opening the Events page.
- Checking that all events appear.
- Opening a correct Event Details link.
- Opening an incorrect event ID.
- Submitting an empty Registration form.
- Entering an invalid email.
- Entering a short or incorrect Student ID.
- Submitting a valid registration.
- Checking that the registration is saved in the CSV file.
- Checking that the registration appears in the table.
- Entering a short Contact message.
- Entering correct Contact form information.
- Opening the website on a smaller screen.

During testing, the website correctly rejected invalid information. When correct information was entered, the registration was saved successfully and appeared in the Registrations List page.

Team Participation

Ruba Almuzaini

Ruba worked on the project idea, sitemap, Home page and shared header and footer. She also helped test the Home and Events pages.

Lamya Alotaibi

Lamya worked on the Events page, Event Details page, event array and PHP loops. She also worked on form validation and tested the event links.

Manal Almusabbihi

Manal worked on the Registration page, CSV storage, Registrations List page, Contact form, CSS and responsive design. She also helped with testing and final documentation.

Mid-Project Progress Report

Work Completed During This Period

During the first three weeks, we selected the idea of Campus Events Hub. We decided to create the website for a fictional University IT Club.

We prepared the sitemap and decided the main pages of the website. These pages were Home, Events, Event Details, Registration, Registrations List and About/Contact.

We also started creating the HTML pages and the main CSS design. We added the navigation bar, event cards, forms and table.

One Key Decision Made

One important decision was to use a PHP array for the events and a CSV file for the registrations.

We selected CSV because the website is small and does not need a complete database system. It was also easier to run using XAMPP without starting MySQL.

One Problem Encountered

One problem was repeating the same header, navigation and footer code on every page.

We solved this problem by creating shared `header.php` and `footer.php` files. We then used `require` to include them on all pages.

Plan for the Next Period

For the next period, we planned to complete the PHP work.

We planned to display the events using loops, create the Event Details page using GET, process the forms using POST, validate the information and save registrations in a CSV file.

We also planned to test all website pages and fix any remaining errors.

Group Members' Participation

Ruba Almuzaini worked on the project idea, sitemap and Home page.

Lamya Alotaibi worked on the Events page, Event Details page and event information.

Manal Almusabbihi worked on the Registration page, CSS, CSV storage and testing.

Progress Log

Week 1

Date: 20 June 2026

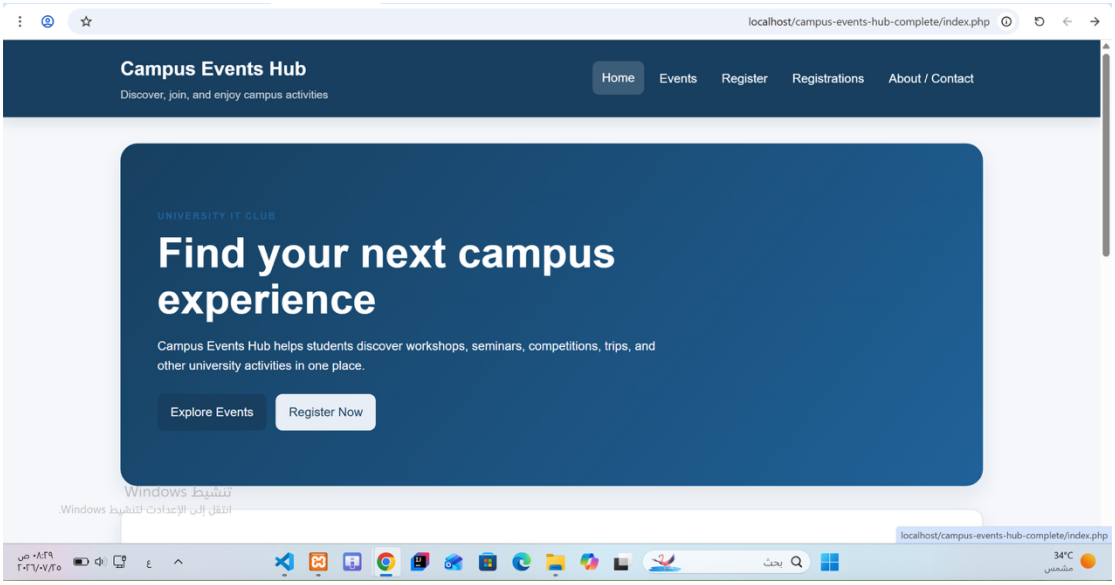
This week, we selected the Campus Events Hub idea and discussed the purpose of the website. We decided to create it for a fictional University IT Club. We also selected the main pages and divided the tasks.

Ruba Almuzaini worked on the project idea and sitemap.

Lamya Alotaibi suggested the main pages and event types.

Manal Almusabbihi helped prepare the first page sketches and divide the tasks.

Screenshot:



Week 2

Date: 27 June 2026

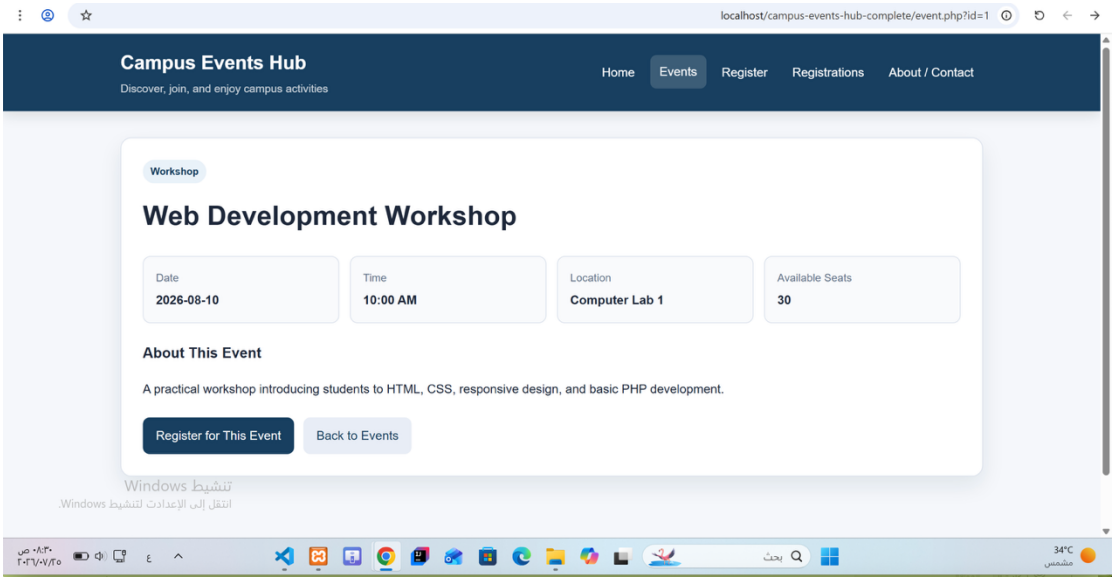
This week, we created the basic HTML pages and connected them using the navigation bar. We prepared the Home, Events, Registration, Registrations List and About/Contact pages.

Ruba Almuzaini worked on the Home page.

Lamya Alotaibi worked on the Events and Event Details pages.

Manal Almusabbihi worked on the Registration and About/Contact pages.

Screenshot:



Week 3

Date: 4 July 2026

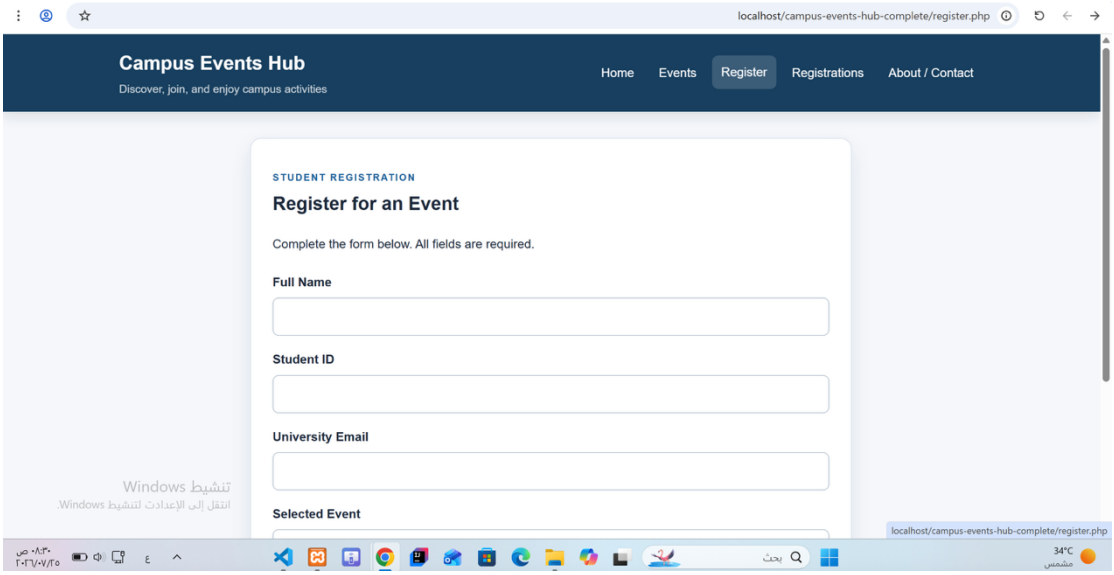
This week, we worked on the CSS design. We added the color scheme, navigation bar, event cards, forms, buttons and table. We also added responsive behavior for small screens.

Ruba Almuzaini worked on the colors and page layout.

Lamya Alotaibi styled the navigation bar and event cards.

Manal Almusabbihi styled the forms, table and responsive design.

Screenshot:



Week 4

Date: 11 July 2026

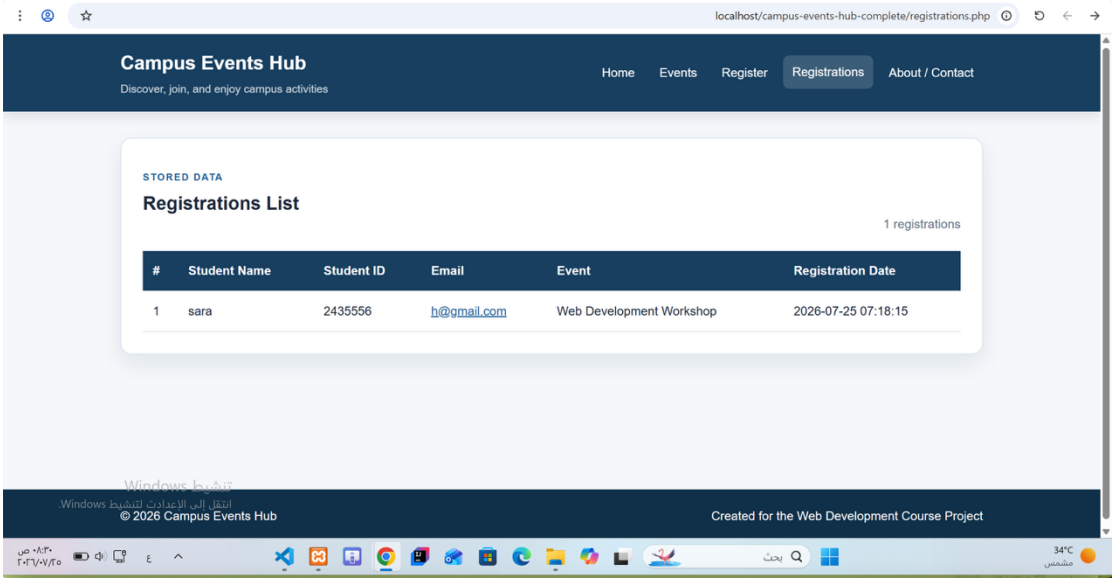
This week, we added PHP to the website. We created shared header and footer files and included them on all pages. We also stored the events in a PHP array and displayed them using loops.

Ruba Almuzaini worked on the shared header and footer.

Lamya Alotaibi added the event array and PHP loops.

Manal Almusabbihi worked on the Event Details page and tested the GET parameter.

Screenshot:



Week 5

Date: 18 July 2026

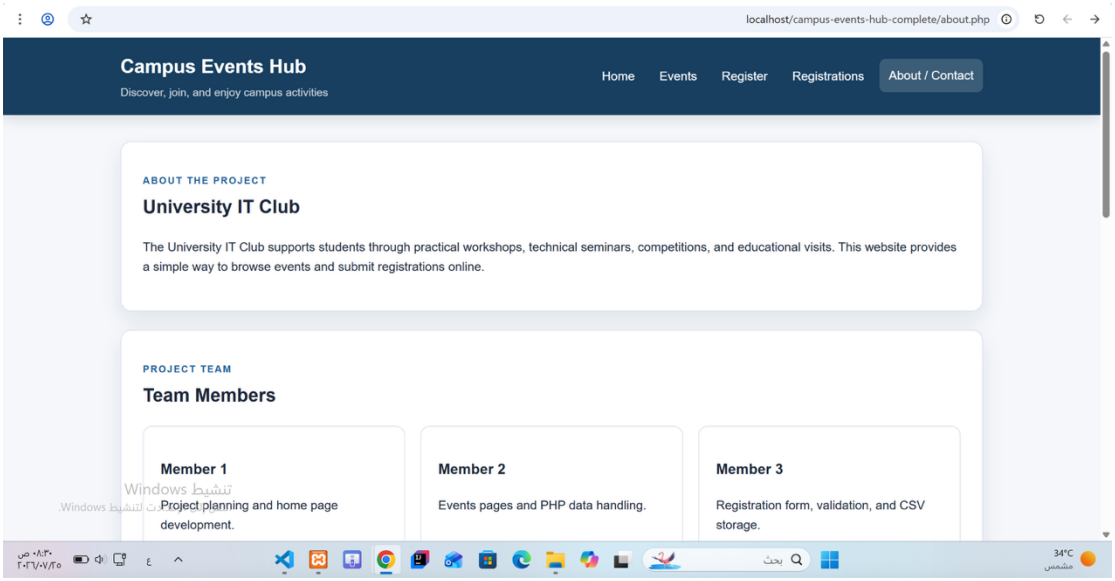
This week, we completed the Registration page and added server-side validation. We saved valid registrations in the CSV file and displayed them in an HTML table.

Ruba Almuzaini worked on the registration form fields.

Lamya Alotaibi worked on form validation.

Manal Almusabbihi worked on CSV saving, CSV reading and the Registrations List page.

Screenshot:



Week 6

Date: 25 July 2026

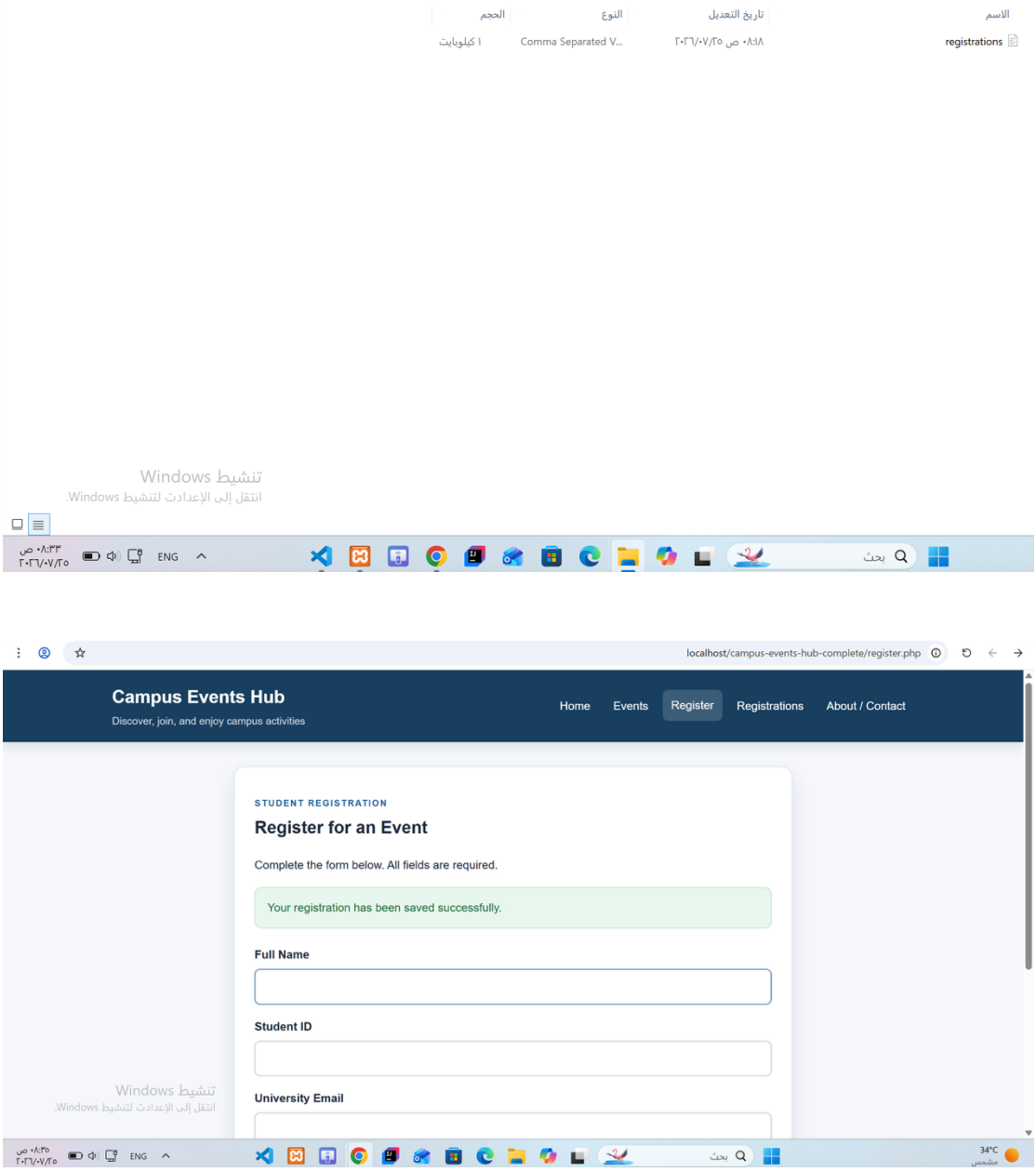
This week, we tested the website and fixed the remaining problems. We tested valid and invalid information, event links, CSV storage, Contact form validation and responsive behavior.

Ruba Almuzaini tested the Home and Events pages.

Lamya Alotaibi tested the Event Details and Registration pages.

Manal Almusabbihi tested the Contact form, CSV file and final project files.

Screenshot:



Project Artifacts

The complete source code is submitted as a ZIP file.

The ZIP file contains:

```
index.php
events.php
event.php
register.php
registrations.php
about.php
includes/header.php
includes/footer.php
includes/events-data.php
css/style.css
data/registrations.csv
```

How to Run

1. Install XAMPP.
2. Extract the project ZIP file.
3. Copy the project folder into:

```
C:\xampp\htdocs\
```

4. Open XAMPP Control Panel.
5. Start Apache.
6. Open a browser.
7. Open:

http://localhost/campus-events-hub-complete/index.php

MySQL is not required because the project uses a CSV file.

Reflection

During this project, we created a dynamic website called Campus Events Hub. The website displays university events and allows students to register for them. At first, the idea seemed simple, but we found that the PHP part required careful testing, especially the forms and CSV storage.

One of the main challenges was saving the registration information correctly. We wanted the data to remain saved even after refreshing or closing the page. First, we validated the form fields using PHP. After that, we used `fputcsv` to save the correct information in the CSV file. We then created the Registrations List page and used PHP to read the same file and display its contents in a table. We tested this by submitting a registration and checking that it appeared in both the CSV file and the table.

We chose CSV instead of MySQL because the project is small and does not contain a large amount of data. CSV was easier to create and run, and it did not require database configuration. We considered using MySQL because it is more organized and more suitable for larger websites. However, it would require creating tables, database connections, and SQL queries. For this project, CSV was enough. The main disadvantage is that CSV is not suitable for a large number of users and does not provide the same level of control as a database.

Another challenge was form validation. The Registration form needed to reject empty fields, invalid email addresses, and incorrect student IDs. The Contact form also needed to reject messages shorter than

ten characters. We tested the forms with invalid information first, and then entered correct information to make sure they worked properly.

We used an artificial intelligence tool only for the permitted purposes, including brainstorming, checking small errors, and reviewing the language and wording of the report. We did not use the output without reviewing it. We tested all website pages ourselves using XAMPP, checked the results, and modified anything that did not match the project requirements.

If we had more time, we would use MySQL and add an administrator login page. We would also allow events to be added and edited through the website, prevent duplicate registrations, and include more images. This project helped us understand how HTML, CSS, and PHP can work together to create a dynamic website.